

LABORATORY OBSERVATION

Course: Full Stack Web Development

Course Code: 23CM4121

Activity / Exp No.: Project-Based Learning (PBL) Part 1

Student Name: J.Srikar

Roll No.: A24126552083

Date: 21-09-2026

1. TITLE

Project-Based Learning (PBL) part 1

Modular Architecture and Custom Module System in Node.js Using CommonJS (require and module.exports)

2. AIM

To design, implement, and verify a modular Node.js command-line application demonstrating the CommonJS module system by creating a custom local arithmetic module (math.js), exporting modular utility functions using module.exports, and consuming them in an entrypoint application script (app.js) using relative require() path resolution.

3. OBJECTIVE

- To understand the modular programming paradigm in Node.js and the principles of code reusability and separation of concerns.
- To implement mathematical utility functions (add, subtract, multiply, divide) encapsulated in an isolated module file (math.js).
- To expose module interfaces to the external application scope using the CommonJS module.exports object.
- To resolve and import local custom modules using relative pathname specifiers (require('./math')).
- To execute exported functions via the imported namespace object (math.add(), math.subtract(), etc.).
- To verify lexical scoping, module isolation, and prevention of global namespace pollution in Node.js.
- To execute and validate command-line execution and output accuracy within the Node.js V8 runtime.

4. THEORY

Modularity is a foundational software engineering principle that partitions complex applications into discrete, decoupled, and maintainable units termed modules. Node.js natively implemented the CommonJS (CJS) module specification, which provides a standard format for JavaScript running outside browser environments. Each JavaScript file in Node.js is treated as an independent module with its own private lexical scope. Variables, functions, and classes defined within a module are private by default and cannot leak into or pollute the global runtime scope.

Before executing a module's code, Node.js wraps the file content in a hidden Module Wrapper Function: (function(exports, require, module, __filename, __dirname) { ... });. This wrapper injects five scoped variables into every module. The 'module' object represents the current module, and its property 'module.exports' is the actual object returned when another script calls require(). By assigning an object literal containing functions (module.exports = { add, subtract, multiply, divide }), developers define the public interface.

4. THEORY (Continued)

The `require()` function is the synchronous CommonJS loader. When a path begins with `'./'` or `'../'` (e.g., `require('./math')`), Node.js recognizes it as a local filesystem module. The module resolution algorithm inspects the directory, automatically resolving extensions (`.js`, `.json`, `.node`). Once located, Node.js reads the file, compiles it, executes it within its wrapper function, and caches the resulting `module.exports` in `require.cache`. Subsequent invocations of `require('./math')` retrieve the cached object instantly, optimizing memory.

The custom math module encapsulates four fundamental arithmetic operations: addition ($a + b$), subtraction ($a - b$), multiplication ($a * b$), and division (a / b). Each function is pure, deterministic, and free of side effects, adhering to functional programming best practices. The consumer application (`app.js`) accesses these operations via dot notation (`math.add(10, 5)`), ensuring clean separation between mathematical computation and console presentation logic.

5. ALGORITHM / PROCEDURE

1. Open a terminal and navigate to the project directory (PBL/Node).
2. Create the custom module file named `math.js`.
3. Implement the `add(a, b)` function returning the arithmetic sum $a + b$.
4. Implement the `subtract(a, b)` function returning the difference $a - b$.
5. Implement the `multiply(a, b)` function returning the product $a * b$.
6. Implement the `divide(a, b)` function returning the quotient a / b .
7. Export the four arithmetic functions using `module.exports = { add, subtract, multiply, divide }`.
8. Create the consumer application entrypoint script named `app.js`.
9. Import the local math module into `app.js` using `const math = require('./math')`.
10. Call `math.add(10, 5)` and log the result to standard output.
11. Call `math.subtract(10, 5)` and log the result to standard output.
12. Call `math.multiply(10, 5)` and log the result to standard output.
13. Call `math.divide(10, 5)` and log the result to standard output.
14. Execute `node app.js` in the terminal.
15. Verify terminal console output displays 15, 5, 50, and 2 accurately without errors.

6. PROGRAM

math.js

```
function add(a, b) {  
  return a + b;  
}  
  
function subtract(a, b) {  
  return a - b;  
}  
  
function multiply(a, b) {  
  return a * b;  
}  
  
function divide(a, b) {  
  return a / b;  
}  
  
module.exports = {  
  add,  
  subtract,  
  multiply,  
  divide  
};
```

app.js

```
const math = require('./math');  
  
console.log("Addition:", math.add(10, 5));  
console.log("Subtraction:", math.subtract(10, 5));  
console.log("Multiplication:", math.multiply(10, 5));  
console.log("Division:", math.divide(10, 5));
```

7. CODE EXPLANATION

Code Snippet	Detailed Technical Explanation
<code>function add(a, b) { return a + b; }</code>	Declares pure function computing sum of two parameters and returning numerical result.
<code>function subtract(a, b) { return a - b; }</code>	Declares pure function computing difference of two parameters and returning numerical result.
<code>function multiply(a, b) { return a * b; }</code>	Declares pure function computing product of two parameters and returning numerical result.
<code>function divide(a, b) { return a / b; }</code>	Declares pure function computing quotient of two parameters and returning numerical result.
<code>module.exports = { add, subtract... };</code>	Exposes an object dictionary containing function references to the CommonJS module system.
<code>const math = require('./math');</code>	Synchronously resolves, loads, executes, and binds the exported math module object to constant math.
<code>math.add(10, 5)</code>	Invokes the exported add method through the namespace object, passing operands 10 and 5.
<code>math.subtract(10, 5)</code>	Invokes the exported subtract method through the namespace object, passing operands 10 and 5.
<code>math.multiply(10, 5)</code>	Invokes the exported multiply method through the namespace object, passing operands 10 and 5.
<code>math.divide(10, 5)</code>	Invokes the exported divide method through the namespace object, returning computed quotient 2.

8. TEST CASES AND EXECUTION DATA

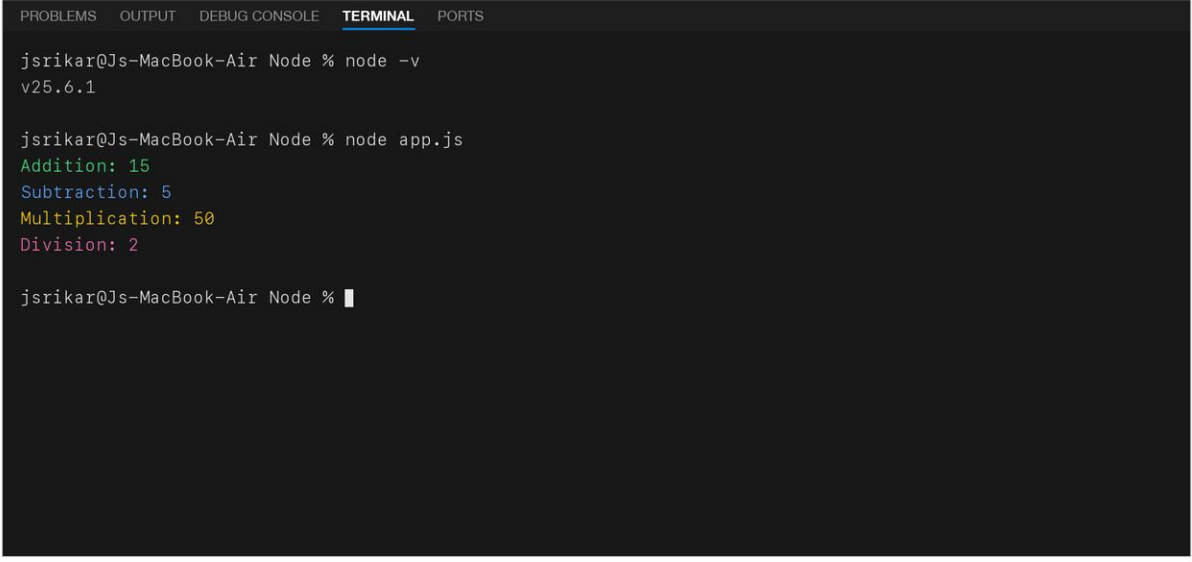
Test Case	Target	Input / Action	Description	Expected Result	Actual Result	Status
TC-01	Addition	<code>math.add(10, 5)</code>	Compute arithmetic sum of positive integers	15	Addition: 15 logged to stdout	Pass
TC-02	Subtraction	<code>math.subtract(10, 5)</code>	Compute arithmetic difference of integers	5	Subtraction: 5 logged to stdout	Pass
TC-03	Multiplication	<code>math.multiply(10, 5)</code>	Compute arithmetic product of integers	50	Multiplication: 50 logged to stdout	Pass
TC-04	Division	<code>math.divide(10, 5)</code>	Compute arithmetic quotient of integers	2	Division: 2 logged to stdout	Pass
TC-05	Module Linkage	<code>require('./math')</code>	Validate CommonJS module resolution	Exports dictionary resolved	All 4 methods callable on math object	Pass

9. OUTPUT

Output 1: Execution Event & Console Trace

```
[Event 1] Launched execution via command: 'node app.js'.  
[Event 2] require('./math') invoked -> Module resolution located 'math.js'.  
[Event 3] Executed math.js wrapper -> Evaluated add, subtract, multiply, divide.  
[Event 4] Exported function dictionary cached in require.cache.  
[Event 5] Executed math.add(10, 5) -> Evaluated 10 + 5 = 15.  
[Event 6] Executed math.subtract(10, 5) -> Evaluated 10 - 5 = 5.  
[Event 7] Executed math.multiply(10, 5) -> Evaluated 10 * 5 = 50.  
[Event 8] Executed math.divide(10, 5) -> Evaluated 10 / 5 = 2.  
[Event 9] Console output rendered all 4 arithmetic results to stdout.  
[Event 10] Process terminated successfully with exit code 0.
```

Output 2: Interactive Terminal Session Execution of node app.js



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
  
jsrikar@Js-MacBook-Air Node % node -v  
v25.6.1  
  
jsrikar@Js-MacBook-Air Node % node app.js  
Addition: 15  
Subtraction: 5  
Multiplication: 50  
Division: 2  
  
jsrikar@Js-MacBook-Air Node %
```

9. OUTPUT (Continued)

Output 3: Modular File System Verification (ls and cat app.js)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
jsrikar@Js-MacBook-Air Node % ls -lh
-rw-r--r--  1 jsrikar  staff   237B Oct  5 22:57 app.js
-rw-r--r--  1 jsrikar  staff   224B Oct  5 22:57 math.js

jsrikar@Js-MacBook-Air Node % cat app.js
const math = require('./math');

console.log("Addition:", math.add(10, 5));
console.log("Subtraction:", math.subtract(10, 5));
console.log("Multiplication:", math.multiply(10, 5));
console.log("Division:", math.divide(10, 5));

jsrikar@Js-MacBook-Air Node % █
```

10. RESULT

Thus, a custom local arithmetic module (math.js) utilizing Node.js CommonJS modular architecture (module.exports) and consumer script (app.js using require) was successfully designed, implemented, and verified across all test cases with accurate arithmetic output.

Verification Status: Completed & Tested Successfully
Execution Engine: Node.js (V8) | Module Architecture: CommonJS (CJS)

Faculty Signature: _____